

Lesson 7: Lists & Grids

Apps are full of lists

Open almost any app and you will see a list: chats in a messaging app, products in a shop, songs in a music player. Showing a scrolling list of items is one of the most common jobs in Flutter.

Flutter gives you `ListView` for this. It is a widget that lays its children out in a scrolling column.

A simple `ListView`

For a short, fixed list, you just put the children inside:

```
ListView(  
  children: const [  
    ListTile(title: Text('Aarav')),  
    ListTile(title: Text('Diya')),  
    ListTile(title: Text('Kabir')),  
  ],  
)
```

`ListTile` is a ready-made row widget: a title with some padding and a divider line, perfect for lists. This works well -- but only for short lists. If you have 1,000 names, Flutter would build all 1,000 rows at once, even the ones you cannot see. That is slow and wasteful.

`ListView.builder` for long lists

For long or changing lists, use `ListView.builder`. Instead of building every row up front, it builds each row only when it is about to appear on screen:

```
final names = ['Aarav', 'Diya', 'Kabir', 'Meera', 'Arjun'];  
  
ListView.builder(  
  itemCount: names.length,  
  itemBuilder: (context, index) {  
    return ListTile(title: Text(names[index]));  
  },  
)
```

Two things to notice:

- `itemCount` -- how many rows there are in total.
- `itemBuilder` -- a function Flutter calls for each row it needs. It gets the row number (`index`), and

you return the widget for that row.

As you scroll, Flutter calls `itemBuilder` for new rows and throws away rows that scroll off screen. A list of 10,000 names uses the same memory as a list of 10. That is why `ListView.builder` is the one you will use most.

A small phonebook example

Here is a full phonebook screen. It keeps the names in a list, builds rows on demand, and shows a phone icon on each row:

```
class Phonebook extends StatelessWidget {
  final contacts = ['Aarav Sharma', 'Diya Patel', 'Kabir Singh', 'Meera Nair'];

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Phonebook')),
      body: ListView.builder(
        itemCount: contacts.length,
        itemBuilder: (context, index) {
          return ListTile(
            leading: const Icon(Icons.person),
            title: Text(contacts[index]),
            onTap: () {
              print('Tapped ${contacts[index]}');
            },
          );
        },
      ),
    );
  }
}
```

`leading` puts an icon at the start of the row, and `onTap` runs when the user taps it. With this pattern you can build almost any list screen.

GridView for grids

At times you want items side by side in a grid -- like photos in a gallery. `GridView` works just like `ListView.builder`, but you also say how many columns you want:

```
GridView.builder(
  gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
    crossAxisCount: 2, // two columns
  ),
  itemCount: 20,
  itemBuilder: (context, index) {
```

```
return Card(  
  child: Center(child: Text('Item $index')),  
);  
,  
)
```

`crossAxisCount` is the number of columns. Everything else -- `itemCount`, `itemBuilder` -- works exactly like the list version. If you know `ListView.builder`, you already know `GridView.builder`.

When to use which

- `ListView` with children -- a short list that never changes (a settings page, a menu).
- `ListView.builder` -- a long list, or a list that can grow or change (contacts, messages, products).
- `GridView.builder` -- the same as above, but shown as a grid (photos, product cards).

When in doubt, reach for the `.builder` version. It is never wrong.

Key terms

- `ListView` -- a scrolling column of widgets.
- `ListTile` -- a ready-made row with a title, made for lists.
- `ListView.builder` -- builds list rows on demand, only when they are about to show.
- `itemCount` -- the total number of rows in a builder list.
- `itemBuilder` -- the function that builds the widget for one row, given its index.
- `GridView` -- like `ListView`, but items are arranged in rows and columns.